NAME:KRISHNAPRIYA P

REG NO:192511279

Course code:CSA0613

Course title: Design and Analysis of Algorithms

CO2_ASSESSMENT TOOL 2_SCENARIO BASED ASSESSMENT

Title: Binary Search – First Element Detection

Aim

To implement the Binary Search algorithm to locate the first element (3) in the sorted dataset [3, 8, 14, 19, 25, 31, 40], count the number of comparisons required, and analyze its time complexity.

Software Used

- Python

Algorithm

- Start.
- Read the sorted array and the target element.
- Initialize low = 0 and high = n - 1.
- Repeat while low <= high.
- Find the middle index using mid = (low + high) // 2.
- Compare the middle element with the target.
- If the middle element equals the target, display the index and stop.
- If the target is smaller than the middle element, update high = mid - 1.
- Otherwise, update low = mid + 1.
- Display the total number of comparisons.
- Stop.

Source Code (Python)

main.py

```python
arr = [3, 8, 14, 19, 25, 31, 40]
target = 3
low = 0
high = len(arr) - 1
comparisons = 0
while low <= high:
    mid = (low + high) // 2
    comparisons += 1
    print("Comparison", comparisons, ": Checking", arr[mid])
    if arr[mid] == target:
        print("\nElement Found")
        print("Index:", mid)
        break
    elif target < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1
else:
    print("Element Not Found")
```

OUTPUT

Output

```
Comparison 1 : Checking 19
Comparison 2 : Checking 8
Comparison 3 : Checking 3

Element Found
Index: 0
Total Comparisons: 3

=== Code Execution Successful ===
```

Analysis

Problem Interpretation

- The given dataset is **[3, 8, 14, 19, 25, 31, 40]**, which is already sorted.
- The objective is to find the element **3** using Binary Search.
- Since Binary Search works efficiently on sorted data, it is the most suitable algorithm for this problem.

Search Process

- **Step 1:** Middle element = **19**, target is smaller → search left half.
- **Step 2:** Middle element = **8**, target is smaller → search left half.
- **Step 3:** Middle element = **3**, target found at **index 0**.

Documentation

- Binary Search repeatedly divides the search interval into two halves.
- Even though the target is the **first element**, the algorithm first checks the middle element before narrowing the search.
- The target element **3** is found after **3 comparisons**.
- The algorithm correctly returns **index 0**, verifying the correctness of the implementation.
- **Time Complexity:**
  - Best Case: **Θ(1)**
  - Average Case: **Θ(log n)**
  - Worst Case: **Θ(log n)**
- **Space Complexity: Θ(1)**